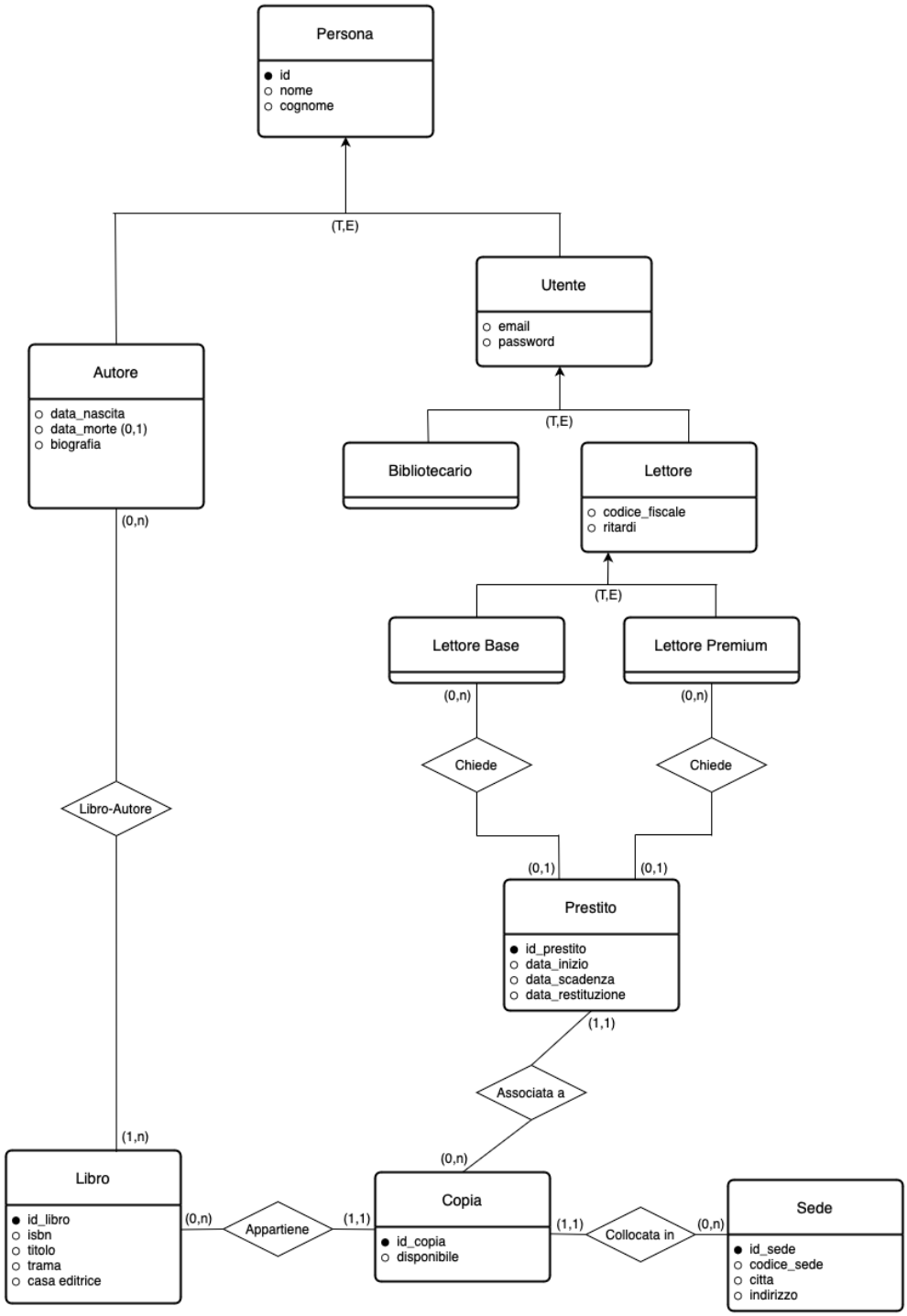
RELAZIONE PROGETTO BASI DI DATI A.A.2024/2025

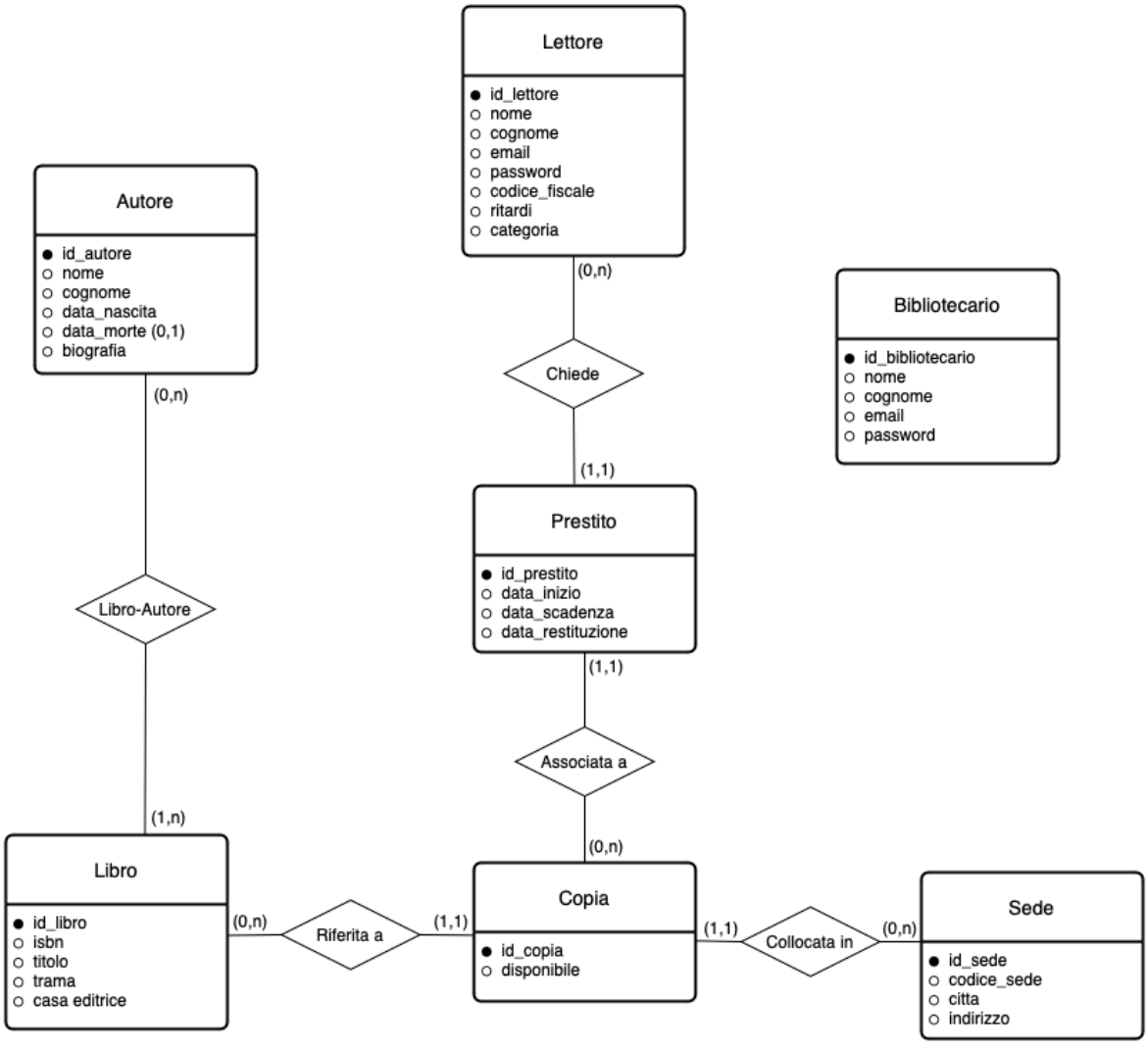
(APPELLO DEL 18 FEBBRAIO 2025)

JACOPO ZAOTTINI 20863A

SCHEMA ER NON RISTRUTTURATO



SCHEMA ER RISTRUTTURATO



SCHEMA RELAZIONALE

Grassetto = chiave primaria

Corsivo = chiave esterna

Asterisco = Cardinalità (0,1) ovvero può essere null

La scelta di aggiungere colonne id di tipo serial e renderle chiavi primarie, anche in casi in cui si poteva usare come chiave primaria un'altra colonna (come nel caso di libro il cui isbn non è stato reso chiave primaria ma semplicemente unique) è dovuta al fatto che, a fronte di poco spazio in più occupato, confronto e indicizzazione vengono velocizzati di molto dato che un serial è un intero, mentre campi come isbn sono varchar, ovvero stringhe, nel nostro caso specifico sempre composte da molti caratteri.

sede (**id_sede**, codice_sede, citta, indirizzo)

autore (**id_autore**, nome, cognome, data_nascita, data_morte*, biografia)

libro (**id_libro**, isbn, titolo, trama, casa_editrice)

libro_autore (***libro***, ***autore***)

copia (**id_copia**, *libro*, *sede*, disponibile)

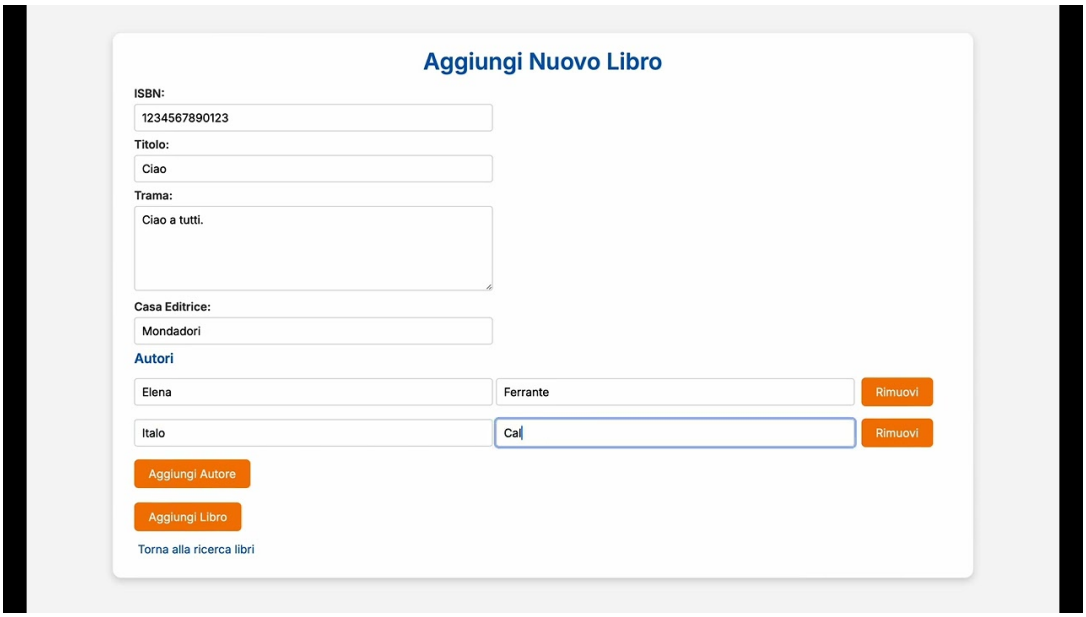
lettore (**id_lettore**, codice_fiscale, nome, cognome, email, password, categoria, ritardi)

bibliotecario (**id_bibliotecario**, nome, cognome, email, password)

prestito (**id_prestito**, *lettore*, *copia*, data_inizio, data_scadenza, data_restituzione*)

Prove di funzionamento

Dato l'elevato numero di funzionalità del sito, ho creato questo video dimostrativo. (Cliccare su di esso).



Nel caso dovessero esserci problemi, il link è il seguente:

https://youtu.be/6B4tKmkzv98/

Manuale Utente

Dotazione software e accesso al sito

- Accertarsi che sul proprio dispositivo siano installati PHP e PostgreSQL.
- Aprire il terminale, spostarsi sulla cartella del progetto ed eseguire il seguente comando (dovesse la porta 9000 essere già in uso, usare un'altra porta):

  ```
  php -S 127.0.0.1:9000
  ```

- Entrare in un browser e nella barra in alto scrivere 127.0.0.1:9000 e premere invio.

CREDENZIALI:

Utente	Email	Password
Lettore 1	marco.rossi@email.com	1234
Lettore 2	giulia.bianchi@email.com	1234
Bibliotecario 1	luca.verdi@email.com	1234
Bibliotecario 2	anna.esposito@email.com	1234

Le funzionalità offerte variano in base al ruolo:

- **Lettore**: utente che consulta il catalogo dei libri, visualizza i dettagli di libri e autori e prenota libri disponibili.
- **Bibliotecario**: utente con diritti di gestione, in grado di modificare i dati, gestire sedi, autori, libri e controllare i prestiti.

1. Accesso alla propria Homepage

Scelta del Tipo Utente

Quando entri nel sito, verrai invitato a scegliere il tipo di utente che desideri utilizzare: lettore o bibliotecario.

Login

Dopo aver selezionato il tipo utente:

- Verrai reindirizzato alla pagina di login.
- Inserisci le credenziali (email e password) per accedere al sistema.
- Al termine del login, verrai indirizzato alla **home** del tuo profilo, con le funzionalità dedicate al tuo ruolo.

2. Funzionalità per il Lettore

Home del Lettore

Una volta autenticato come Lettore, potrai accedere alle seguenti funzionalità:

- **Ricerca Libri**:
  - Usa la barra di ricerca per cercare libri per titolo o ISBN.
- **Visualizzazione Dettagli Libro**:
  - Cliccando su un libro nei risultati di ricerca, verrai reindirizzato alla pagina di dettaglio.
  - Qui potrai leggere la trama e vedere chi sono gli autori.

- **Prenotazione**:
  - Se il libro è disponibile, potrai selezionare la sede presso cui desideri effettuare la prenotazione e prenotarlo cliccando sul pulsante “Prenota”.
- **Ricerca Autori**:
  - Usa la barra di ricerca per cercare autori per nome o cognome.
- **Visualizzazione Dettagli Autore**:
  - Cliccando su un autore nei risultati di ricerca, verrai reindirizzato alla pagina di dettaglio.
  - Qui potrai leggere i dettagli dell'autore e quali libri ha scritto.
- **Storico Prestiti**:
  - Visualizzare i tuoi prestiti e il relativo stato.

Altro

- **Modifica profilo**: Qui puoi modificare la password o gli altri dati del tuo profilo.
- **Logout**: Ricordati di effettuare il logout al termine della sessione.

3. Funzionalità per il Bibliotecario

Home del Bibliotecario

Accedendo come Bibliotecario, avrai a disposizione un pannello di controllo con funzionalità avanzate di gestione:

- **Ricerca e gestione Libri**:
  - Cerca i libri presenti nel catalogo e accedi alla pagina di dettaglio di ciascun libro.
  - Modifica i dati dei libri (titolo, trama, casa editrice, ecc.) e gestisci le informazioni sulle copie.
- **Ricerca e gestione Lettori**:
  - Potrai visualizzare l’elenco dei lettori e gestire i loro profili.
- **Ricerca e gestione Sedi**:
  - Visualizza le informazioni sulle sedi e il numero di copie totali e disponibili per ciascuna sede.
  - Aggiungi o rimuovi copie tramite le apposite funzionalità, tenendo conto delle regole (ad es. non rimuovere copie se sono in prestito).
- **Ricerca e gestione Autori**:
  - Visualizza i dettagli degli autori.
  - Modifica i dati degli autori (nome, data di nascita/morte, biografia).
  - Associa o rimuovi libri agli autori, garantendo che la rimozione non comprometta l’integrità del catalogo (ad esempio, un libro deve avere almeno un autore).
- **Eliminazione di Autori**:
  - Se necessario, puoi eliminare un autore **solo** se non risulta essere l’unico autore di nessun libro (il sistema verifica automaticamente tale condizione).

- **Gestione Prestiti**:
  - Monitora i prestiti attivi e quelli conclusi.
  - Potrai aggiornare la data di scadenza per i prestiti o segnare una copia come restituita.

Altro

- **Statistiche**: In questa sezione puoi visualizzare le statistiche per sede e i ritardi per sede.
- **Modifica profilo**: Qui puoi modificare la password o gli altri dati del tuo profilo.
- **Logout**: Alla fine della sessione, ricordati di effettuare il logout.

Funzioni e Trigger

Controlla condizioni prestito

```
CREATE OR REPLACE FUNCTION controlla_condizioni_prestito()
RETURNS TRIGGER AS $$
DECLARE
    cnt_ritardi INTEGER;
    cat categoria_lettore;
    cnt_attivi INTEGER;
    available BOOLEAN;
BEGIN
    -- Verifica il contatore dei ritardi e la categoria del lettore
    SELECT ritardi, categoria INTO cnt_ritardi, cat
      FROM lettore
     WHERE id_lettore = NEW.lettore;

    IF cnt_ritardi >= 5 THEN
        RAISE EXCEPTION 'Il lettore ha % ritardi, non può richiedere nuovi prestiti', cnt_ritardi;
    END IF;

    -- Conta i prestiti attivi (senza restituzione) per il lettore
    SELECT COUNT(*) INTO cnt_attivi FROM prestito
     WHERE lettore = NEW.lettore AND data_restituzione IS NULL;

    IF cat = 'base' AND cnt_attivi >= 3 THEN
        RAISE EXCEPTION 'Il lettore è di categoria base e ha già % prestiti attivi', cnt_attivi;
    ELSIF cat = 'premium' AND cnt_attivi >= 5 THEN
        RAISE EXCEPTION 'Il lettore è di categoria premium e ha già % prestiti attivi', cnt_attivi;
    END IF;

    -- Verifica che la copia richiesta sia disponibile
    SELECT disponibile INTO available FROM copia WHERE id_copia = NEW.copia;

    IF NOT available THEN
        RAISE EXCEPTION 'La copia % non è disponibile', NEW.copia;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_controlla_condizioni_prestito
BEFORE INSERT ON prestito
FOR EACH ROW EXECUTE FUNCTION controlla_condizioni_prestito();
```

La funzione controlla_condizioni_prestito verifica che un prestito possa essere assegnato.
Essa controlla che il lettore abbia meno di 5 ritardi.
Poi controlla che il lettore non supererebbe un numero di prestiti attivi di 3 se è utente base, 5 se è utente premium.
Dopo controlla che la copia richiesta sia disponibile.

Il trigger trg_controlla_condizioni_prestito chiama tale funzione prima dell'inserimento di un nuovo prestito.

Segna copia non disponibile

```
CREATE OR REPLACE FUNCTION segna_copia_non_disponibile()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE copia
       SET disponibile = FALSE
     WHERE id_copia = NEW.copia;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_segna_copia_non_disponibile
AFTER INSERT ON prestito
FOR EACH ROW EXECUTE FUNCTION segna_copia_non_disponibile();
```

La funzione segna_copia_non_disponibile segna una copia come non disponibile.

Il relativo trigger chiama tale funzione dopo un inserimento nella tabella prestito.

Refresh sede statistiche

```
-- Funzione che refresha la materialized view sede_statistiche
CREATE OR REPLACE FUNCTION refresh_sede_statistiche()
RETURNS trigger AS $$
BEGIN
    REFRESH MATERIALIZED VIEW sede_statistiche;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Trigger che si attiva con azione su prestito
CREATE TRIGGER trg_refresh_prestiti
AFTER INSERT OR UPDATE OR DELETE ON prestito
FOR EACH STATEMENT
EXECUTE FUNCTION refresh_sede_statistiche();

-- Trigger che si attiva con azione su copia
CREATE TRIGGER trg_refresh_copie
AFTER INSERT OR UPDATE OR DELETE ON copia
FOR EACH STATEMENT
EXECUTE FUNCTION refresh_sede_statistiche();
```

La funzione refresh_sede_statistiche fa un refresh della materialized view sede_statistiche.

I relativi trigger chiamano tale funzione dopo un'azione sulla tabella prestito o sulla tabella copia.

Procedure

Proroga prestito

```
CREATE OR REPLACE PROCEDURE proroga_prestito(p_id INTEGER, p_new_scadenza DATE)
LANGUAGE plpgsql
AS $$
DECLARE
    current_scadenza DATE;
BEGIN
    -- Recupera la data di scadenza corrente per il prestito indicato
    SELECT data_scadenza INTO current_scadenza
      FROM prestito
     WHERE id_prestito = p_id;

    IF current_scadenza IS NULL THEN
        RAISE EXCEPTION 'Prestito non trovato';
    END IF;

    -- Se il prestito è già in ritardo, non è consentita la proroga
    IF CURRENT_DATE > current_scadenza THEN
        RAISE EXCEPTION 'Il prestito è già in ritardo e non può essere prorogato';
    END IF;

    -- Se la nuova data non è successiva all'attuale data di scadenza, solleva un'eccezione
    IF p_new_scadenza <= current_scadenza THEN
        RAISE EXCEPTION 'La nuova scadenza deve essere successiva all’attuale data di scadenza';
    END IF;

    -- Aggiorna la data di scadenza con il nuovo valore
    UPDATE prestito
       SET data_scadenza = p_new_scadenza
     WHERE id_prestito = p_id;
END;
$$;
```

La procedura proroga_prestito effettua la proroga di un prestito, andando ad aggiornare la data di scadenza qualora il prestito non sia già in ritardo e qualora la nuova data di scadenza sia successiva alla data di scadenza attuale.

Assegna prestito

```
CREATE OR REPLACE PROCEDURE assegna_prestito(
    p_id_lettore INTEGER,
    p_id_libro INTEGER,
    p_id_sede INTEGER DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_id_copia INTEGER;
    available_sedi TEXT;
BEGIN
    IF p_id_sede IS NULL THEN
        -- Nessuna sede scelta: seleziona una copia disponibile in qualunque sede
        SELECT id_copia INTO v_id_copia
        FROM copia
        WHERE libro = p_id_libro AND disponibile = TRUE
        LIMIT 1;

        IF v_id_copia IS NULL THEN
           RAISE EXCEPTION 'Il libro non è disponibile in nessuna sede.';
        END IF;
    ELSE
        -- Sede scelta: cerca una copia disponibile in quella sede
        SELECT id_copia INTO v_id_copia
        FROM copia
        WHERE libro = p_id_libro AND disponibile = TRUE AND sede = p_id_sede
        LIMIT 1;

        IF v_id_copia IS NULL THEN
            -- Se non c'è copia disponibile in quella sede, recupera l'elenco delle sedi dove il libro è disponibile
            SELECT string_agg(s.codice_sede, ', ') INTO available_sedi
            FROM copia c JOIN sede s ON c.sede = s.id_sede
            WHERE c.libro = p_id_libro AND c.disponibile = TRUE;

            IF available_sedi IS NULL THEN
                RAISE EXCEPTION 'Il libro non è disponibile né in questa sede né in nessun altra.';
            ELSE
                RAISE EXCEPTION 'Il libro non è disponibile nella sede scelta. Disponibile nelle sedi: %', available_sedi;
            END IF;
        END IF;
    END IF;

    -- Inserimento del prestito
    INSERT INTO prestito (lettore, copia)
    VALUES (p_id_lettore, v_id_copia);

    -- Aggiornamento della disponibilità della copia assegnata
    UPDATE copia SET disponibile = FALSE WHERE id_copia = v_id_copia;
END;
$$;
```

La procedura assegna_prestito, se non è stata scelta alcuna sede seleziona una copia disponibile in una sede qualunque.

Se invece è stata scelta una sede, seleziona una copia disponibile in tale sede (se lì non c'è una copia disponibile del libro, ma c'è in almeno un'altra sede, lancia un'eccezione in cui menziona le sedi in cui la copia è presente).

Dopo aver assegnato con successo il prestito al lettore, setta la copia come non disponibile.

Azzera ritardi

```
CREATE OR REPLACE PROCEDURE azzera_ritardi(lettore_target INTEGER)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE lettore
       SET ritardi = 0
     WHERE id_lettore = lettore_target;

END;
$$;
```

La procedura azzera_ritardi va ad azzerare il numero di ritardi associati a un lettore.

Views e Materialized Views

Materialized view sede statistiche

```
CREATE MATERIALIZED VIEW sede_statistiche AS
SELECT
    s.codice_sede,
    s.citta,
    s.indirizzo,
    COUNT(c.id_copia) AS totale_copie,
    COUNT(DISTINCT c.libro) AS totale_isbn,
    SUM(CASE WHEN p.id_prestito IS NOT NULL AND p.data_restituzione IS NULL THEN 1 ELSE 0 END)
AS prestiti_in_corso
FROM sede s
LEFT JOIN copia c ON s.id_sede = c.sede
LEFT JOIN prestito p ON c.id_copia = p.copia
GROUP BY s.codice_sede, s.citta, s.indirizzo;
```

La materialized view sede_statistiche mostra, per ogni sede, il codice associato, la città dove si trova, l'indirizzo, il numero di copie in sua gestione, il numero di isbn e il numero di prestiti in corso.

Ho scelto di usare una materialized view poiché le statistiche per ogni sede cambiano solo quando vengono effettuate azioni su prestito o copia, il che non avviene molto spesso, mentre un bibliotecario può accedere alla pagina web che mostra queste statistiche molto più spesso. In questo modo non andiamo a far eseguire la query ogni volta.

View sede ritardi

```sql
CREATE OR REPLACE VIEW sede_ritardi AS
SELECT
    s.codice_sede,
    s.citta,
    s.indirizzo,
    lib.isbn,
    lib.titolo,
    p.id_prestito,
    p.data_inizio,
    p.data_scadenza,
    lett.codice_fiscale,
    lett.nome as nome_lettore,
    lett.cognome as cognome_lettore
FROM prestito p
JOIN copia c ON p.copia = c.id_copia
JOIN sede s ON c.sede = s.id_sede
JOIN libro lib ON c.libro = lib.id_libro
JOIN lettore lett ON p.lettore = lett.id_lettore
WHERE p.data_restituzione IS NULL
  AND CURRENT_DATE > p.data_scadenza;
```

La view sede_ritardi mostra tutti i prestiti in ritardo per ogni sede e varie relative informazioni.

Ho scelto di usare una view poiché la query non ha una complessità temporale eccessivamente elevata e poiché se avessi usato una materialized view avrei dovuto aggiornarla (oltre a quando un prestito viene restituito) ogni giorno a mezzanotte (momento in cui nuovi prestiti diventano in ritardo). Se in futuro si optasse per scadenze di prestiti associate a timestamp, andrebbero effettuati check a intervalli regolari e il numero di refresh necessari diventerebbe molto più alto. Quindi, per evitare problemi legati a ciò, ho optato per la soluzione più semplice, anche se l'alternativa è senz'altro accettabile.